Spring Boot Gen AI with Gradle Groovy Setup Guide

Project Structure

```
spring-boot-genai/
├── gradle/
│   └── wrapper/
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/genai/
│   │   │       ├── GenAIApplication.java
│   │   │       ├── config/
│   │   │       │   ├── AIConfiguration.java
│   │   │       │   └── AIProperties.java
│   │   │       ├── controller/
│   │   │       │   └── GenAIController.java
│   │   │       ├── dto/
│   │   │       │   ├── AIRequest.java
│   │   │       │   └── AIResponse.java
│   │   │       └── service/
│   │   │           └── GenAIService.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
│       └── java/
│           └── com/genai/
│               └── GenAIApplicationTests.java
├── build.gradle
├── settings.gradle
├── gradle.properties
├── gradlew
├── gradlew.bat
└── README.md
```

Installation & Setup

Step 1: Create Project Directory

```bash
mkdir spring-boot-genai
cd spring-boot-genai
```

Step 2: Initialize Gradle Project

```bash
gradle init --type java-application
```

Step 3: Copy Files

Copy all the provided files into your project:

- `build.gradle`
- `settings.gradle`
- `gradle.properties`
- `gradle/wrapper/gradle-wrapper.properties`

Step 4: Create Java Package Structure

```bash
mkdir -p src/main/java/com/genai/{config,controller,dto,service}
mkdir -p src/main/resources
mkdir -p src/test/java/com/genai
```

Step 5: Create Source Files

Place all Java files in appropriate directories:

- `src/main/java/com/genai/GenAIApplication.java`
- `src/main/java/com/genai/config/AIConfiguration.java`
- `src/main/java/com/genai/config/AIProperties.java`
- `src/main/java/com/genai/controller/GenAIController.java`

- `src/main/java/com/genai/dto/AIRequest.java`
- `src/main/java/com/genai/dto/AIResponse.java`
- `src/main/java/com/genai/service/GenAIService.java`

Step 6: Copy Configuration Files

```bash
cp application.properties src/main/resources/
```

Using Gradle

Build the Project

```bash
./gradlew build
```

Run the Application

```bash
./gradlew bootRun
```

Run Tests

```bash
./gradlew test
```

Clean Build

```bash
./gradlew clean
```

View Dependencies

```bash
./gradlew dependencies
```

Generate IDE Configuration (IntelliJ IDEA)

```bash
./gradlew idea
```

Generate IDE Configuration (Eclipse)

```bash
./gradlew eclipse
```

Environment Variables

Linux/Mac

```bash
export GOOGLE_GEMINI_API_KEY=your-api-key-here
./gradlew bootRun
```

Windows (PowerShell)

```powershell
$env:GOOGLE_GEMINI_API_KEY="your-api-key-here"
./gradlew bootRun
```

Windows (CMD)

```cmd
set GOOGLE_GEMINI_API_KEY=your-api-key-here
gradlew bootRun
```

Configuration

Update `src/main/resources/application.properties`:

```properties
```

```
# Server Configuration
server.port=8080
server.servlet.context-path=/api

# Application Name
spring.application.name=Spring Boot Gen AI with Google Gemini

# Google Gemini Configuration
spring.ai.google.gemini.api-key=${GOOGLE_GEMINI_API_KEY:your-api-key-here}
spring.ai.google.gemini.project-id=${GOOGLE_PROJECT_ID:your-project-id}

# Model Configuration
app.ai.model.name=gemini-1.5-pro
app.ai.model.temperature=0.7
app.ai.model.max-tokens=1000
app.ai.model.top-p=0.95
app.ai.model.top-k=40

# Logging
logging.level.root=INFO
logging.level.com.genai=DEBUG
```

API Endpoints

1. Generate Response

bash

```
curl -X POST http://localhost:8080/api/v1/ai/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt":"What is Spring Boot?"}'
```

2. Summarize Text

bash

```
curl -X POST http://localhost:8080/api/v1/ai/summarize \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Your long text here..."}'
```

3. Answer Question

```bash
curl -X POST http://localhost:8080/api/v1/ai/question \
  -H "Content-Type: application/json" \
  -d '{"prompt":"What is Gradle?"}'
```

4. Translate Text

```bash
curl -X POST http://localhost:8080/api/v1/ai/translate \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Hello World","targetLanguage":"Spanish"}'
```

5. Generate Code

```bash
curl -X POST http://localhost:8080/api/v1/ai/generate-code \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Create a Java method to calculate factorial"}'
```

6. Analyze Text

```bash
curl -X POST http://localhost:8080/api/v1/ai/analyze \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Your text to analyze..."}'
```

7. Health Check

```bash
curl http://localhost:8080/api/v1/ai/health
```

8. Model Info

```bash
curl http://localhost:8080/api/v1/ai/model-info
```

Gradle Commands Cheat Sheet

Command	Description
`./gradlew build`	Build the project
`./gradlew bootRun`	Run Spring Boot application
`./gradlew test`	Run tests
`./gradlew clean`	Clean build directory
`./gradlew dependencies`	Show project dependencies
`./gradlew tasks`	List all tasks
`./gradlew --version`	Show Gradle version
`./gradlew jar`	Build JAR file
`./gradlew bootJar`	Build executable JAR

Troubleshooting

1. Gradle Wrapper Permission Issue (Linux/Mac)

```bash
chmod +x gradlew
```

2. Java Version Mismatch

Ensure Java 17+ is installed:

```bash
java -version
```

3. Clear Gradle Cache

```bash
```

```
./gradlew clean --refresh-dependencies
```

4. API Key Not Found

Ensure environment variable is set before running:

```bash
export GOOGLE_GEMINI_API_KEY=your-key
./gradlew bootRun
```

IDE Integration

IntelliJ IDEA

1. Open project
2. IDE automatically detects Gradle configuration
3. Gradle tool window appears on right side
4. Run tasks from there

Eclipse

```bash
./gradlew eclipse
```

Then import project in Eclipse

VS Code

Install "Gradle for Java" extension from marketplace

Additional Resources

- Spring Boot Documentation
- Gradle Documentation
- Spring AI Documentation
- Google Gemini API